CLAUDE.md — NerdCommand.info Site Intelligence

> Claude Code reads this file automatically on project open.
> This file is the single source of truth for site identity, GitHub location, update procedures, and monitoring config.

Project Identity

Field	Value
Site Name	NerdCommand.info
Organization	GangsterNerds LLC
GitHub Repo	`https://github.com/YOUR_USERNAME/nerdcommand-site`
GitHub Owner	`YOUR_USERNAME`
GitHub Repo Name	`nerdcommand-site`
Default Branch	`main`
Live URL	`https://nerdcommand.info`
Primary File	`index.html`
Config File	`site_config.json`
Manager Script	`site_manager.py`

> **FIRST THING:** Replace `YOUR_USERNAME` above and in `site_config.json` with the actual GitHub username before running anything.

What You Can Ask Claude to Do

Updates

- "Update the pricing section to add a new tier at $299/month"
- "Change the hero headline to [new text]"

- "Add a new division card for [division name]"
- "Update the CTA email to [new email]"
- "Push the current index.html to GitHub"

Monitoring

- "Run a site health check"
- "Start the monitor — check every 30 minutes"
- "Show me the last monitor report"
- "Check if the contact form link is working"

Deployment

- "Deploy latest index.html to GitHub"
- "Show me what changed since last push"
- "Roll back to the previous version"

File Map

```
nerdcommand-site/
├── CLAUDE.md           ← You are here. Claude reads this first.
├── index.html          ← The live website (single file)
├── site_config.json    ← Repo location + monitoring settings
├── site_manager.py     ← Update + monitor tool (Python)
├── CNAME               ← Contains: nerdcommand.info
├── monitor_log.json    ← Auto-generated monitoring history
└── .github/
    └── workflows/
        └── deploy.yml  ← Auto-deploy on push
```

---

site_config.json

Create this file in the repo root with your values filled in:

```json
```

```
{
  "project": "nerdcommand-site",
  "organization": "GangsterNerds LLC",
  "github": {
    "owner": "YOUR_USERNAME",
    "repo": "nerdcommand-site",
    "branch": "main",
    "primary_file": "index.html",
    "api_base": "https://api.github.com"
  },
  "site": {
    "live_url": "https://nerdcommand.info",
    "fallback_url": "https://YOUR_USERNAME.github.io/nerdcommand-site",
    "contact_email": "hello@nerdcommand.info"
  },
  "monitoring": {
    "enabled": true,
    "interval_minutes": 30,
    "checks": [
      "uptime",
      "response_time",
      "page_title",
      "contact_link",
      "ssl"
    ],
    "alert_threshold_ms": 3000,
    "log_file": "monitor_log.json"
  },
  "sections": {
    "hero": "class=\"hero\"",
    "divisions": "id=\"divisions\"",
    "agent": "id=\"agent\"",
    "trading": "id=\"trading\"",
    "pricing": "id=\"pricing\"",
    "cta": "id=\"cta\""
  }
}
```

---

site_manager.py

Create this file in the repo root:

python

```
#!/usr/bin/env python3
"""
NerdCommand Site Manager
Handles GitHub updates, deployment, and uptime monitoring.
Usage:
  python site_manager.py push          — push index.html to GitHub
  python site_manager.py check         — run one health check
  python site_manager.py monitor       — run continuous monitor loop
  python site_manager.py fetch         — pull current index.html from GitHub
  python site_manager.py log           — print last 10 monitor entries
"""

import os, sys, json, time, base64, hashlib, datetime
import urllib.request, urllib.error

# ── CONFIG ──────────────────────────────────────────────────────────────

def load_config(path="site_config.json"):
    with open(path) as f:
        return json.load(f)

def github_headers(token):
    return {
        "Authorization": f"token {token}",
        "Accept": "application/vnd.github.v3+json",
        "Content-Type": "application/json",
        "User-Agent": "NerdCommand-SiteManager/1.0"
    }

def now():
    return datetime.datetime.utcnow().isoformat() + "Z"

# ── GITHUB API ──────────────────────────────────────────────────────────

def gh_get(url, token):
    req = urllib.request.Request(url, headers=github_headers(token))
    with urllib.request.urlopen(req) as r:
        return json.loads(r.read())

def gh_put(url, token, payload):
    data = json.dumps(payload).encode()
    req = urllib.request.Request(url, data=data, headers=github_headers(token), meth
    with urllib.request.urlopen(req) as r:
```

```
        return json.loads(r.read())

def get_file_sha(cfg, token, filename):
    """Get current SHA of a file in the repo (required for updates)."""
    c = cfg["github"]
    url = f"{c['api_base']}/repos/{c['owner']}/{c['repo']}/contents/{filename}?ref={
    try:
        result = gh_get(url, token)
        return result.get("sha")
    except urllib.error.HTTPError as e:
        if e.code == 404:
            return None  # File doesn't exist yet
        raise

# ── PUSH ──────────────────────────────────────────────────────────────

def push_file(cfg, token, filename="index.html", message=None):
    """Push a local file to GitHub."""
    if not os.path.exists(filename):
        print(f"❌ {filename} not found locally.")
        return False

    with open(filename, "rb") as f:
        content = f.read()

    encoded = base64.b64encode(content).decode()
    sha = get_file_sha(cfg, token, filename)
    commit_msg = message or f"Site update via NerdCommand Manager — {now()}"

    c = cfg["github"]
    url = f"{c['api_base']}/repos/{c['owner']}/{c['repo']}/contents/{filename}"
    payload = {
        "message": commit_msg,
        "content": encoded,
        "branch": c["branch"]
    }
    if sha:
        payload["sha"] = sha

    result = gh_put(url, token, payload)
    commit_url = result.get("commit", {}).get("html_url", "")
    print(f"✅ Pushed {filename} to GitHub")
    print(f"   Commit: {commit_url}")
    return True
```

```
# ── FETCH ──────────────────────────────────────────────────────────────

def fetch_file(cfg, token, filename="index.html"):
    """Pull current file from GitHub to local."""
    c = cfg["github"]
    url = f"{c['api_base']}/repos/{c['owner']}/{c['repo']}/contents/{filename}?ref={
    result = gh_get(url, token)
    content = base64.b64decode(result["content"]).decode("utf-8")
    with open(filename, "w", encoding="utf-8") as f:
        f.write(content)
    sha = result["sha"][:7]
    print(f"✅ Fetched {filename} from GitHub (sha: {sha})")
    return content

# ── HEALTH CHECK ───────────────────────────────────────────────────────

def health_check(cfg):
    """Run a full site health check. Returns a report dict."""
    url = cfg["site"]["live_url"]
    report = {
        "timestamp": now(),
        "url": url,
        "checks": {}
    }

    # Uptime + response time
    try:
        start = time.time()
        req = urllib.request.Request(url, headers={"User-Agent": "NerdCommand-Monito
        with urllib.request.urlopen(req, timeout=10) as r:
            elapsed_ms = int((time.time() - start) * 1000)
            html = r.read().decode("utf-8", errors="ignore")
            status = r.status

        report["checks"]["uptime"] = {"status": "ok", "http_code": status}
        report["checks"]["response_time"] = {
            "status": "ok" if elapsed_ms < cfg["monitoring"]["alert_threshold_ms"] e
            "ms": elapsed_ms
        }

        # Page title check
        if "NerdCommand" in html:
            report["checks"]["page_title"] = {"status": "ok"}
```

```
        else:
            report["checks"]["page_title"] = {"status": "warn", "note": "NerdCommand

        # Contact link check
        email = cfg["site"]["contact_email"]
        if email in html:
            report["checks"]["contact_link"] = {"status": "ok", "email": email}
        else:
            report["checks"]["contact_link"] = {"status": "warn", "note": f"{email}

        # Division sections check
        sections_ok = all(v.replace('"', '') in html for v in cfg["sections"].values
        report["checks"]["sections"] = {
            "status": "ok" if sections_ok else "warn",
            "note": "All 6 sections found" if sections_ok else "Some sections missin
        }

    except urllib.error.URLError as e:
        report["checks"]["uptime"] = {"status": "down", "error": str(e)}
    except Exception as e:
        report["checks"]["uptime"] = {"status": "error", "error": str(e)}

    # Overall status
    statuses = [v.get("status") for v in report["checks"].values()]
    if "down" in statuses or "error" in statuses:
        report["overall"] = "🔴 DOWN"
    elif "warn" in statuses or "slow" in statuses:
        report["overall"] = "🟡 DEGRADED"
    else:
        report["overall"] = "🟢 HEALTHY"

    return report

def print_report(report):
    print(f"\n{'='*50}")
    print(f"  NerdCommand Site Monitor — {report['timestamp']}")
    print(f"  {report['overall']}  |  {report['url']}")
    print(f"{'='*50}")
    for check, data in report["checks"].items():
        status_icon = {"ok": "✅", "warn": "⚠️ ", "slow": "🐢", "down": "🔴", "erro
        detail = ""
        if "ms" in data:
            detail = f"  ({data['ms']}ms)"
        elif "note" in data:
```

```
            detail = f"  — {data['note']}"
        elif "error" in data:
            detail = f"  — {data['error']}"
        print(f"  {status_icon} {check.replace('_',' ').title()}{detail}")
    print()

def log_report(report, log_file="monitor_log.json"):
    """Append report to the monitor log file."""
    history = []
    if os.path.exists(log_file):
        with open(log_file) as f:
            try:
                history = json.load(f)
            except Exception:
                history = []
    history.append(report)
    history = history[-200:]  # Keep last 200 entries
    with open(log_file, "w") as f:
        json.dump(history, f, indent=2)

# ── MONITOR LOOP ──────────────────────────────────────────────────────

def monitor_loop(cfg):
    interval = cfg["monitoring"]["interval_minutes"] * 60
    print(f"🔁 NerdCommand Monitor started — checking every {cfg['monitoring']['inte
    print(f"   Target: {cfg['site']['live_url']}")
    print(f"   Press Ctrl+C to stop\n")
    while True:
        report = health_check(cfg)
        print_report(report)
        log_report(report, cfg["monitoring"]["log_file"])
        time.sleep(interval)

# ── LOG VIEWER ────────────────────────────────────────────────────────

def show_log(cfg, n=10):
    log_file = cfg["monitoring"]["log_file"]
    if not os.path.exists(log_file):
        print("No monitor log found. Run: python site_manager.py monitor")
        return
    with open(log_file) as f:
        history = json.load(f)
    recent = history[-n:]
    print(f"\nLast {len(recent)} monitor entries:\n")
```

```
    for entry in recent:
        ms = entry.get("checks", {}).get("response_time", {}).get("ms", "—")
        print(f"  {entry['overall']}  {entry['timestamp']}  {ms}ms")
    print()

# ── MAIN ──────────────────────────────────────────────────────────────

def main():
    cfg = load_config()
    token = os.environ.get("GITHUB_TOKEN")

    cmd = sys.argv[1] if len(sys.argv) > 1 else "help"

    if cmd in ("push", "fetch") and not token:
        print("❌ GITHUB_TOKEN environment variable not set.")
        print("   export GITHUB_TOKEN=your_personal_access_token")
        sys.exit(1)

    if cmd == "push":
        msg = sys.argv[2] if len(sys.argv) > 2 else None
        push_file(cfg, token, message=msg)

    elif cmd == "fetch":
        fetch_file(cfg, token)

    elif cmd == "check":
        report = health_check(cfg)
        print_report(report)
        log_report(report, cfg["monitoring"]["log_file"])

    elif cmd == "monitor":
        monitor_loop(cfg)

    elif cmd == "log":
        show_log(cfg)

    else:
        print(__doc__)

if __name__ == "__main__":
    main()
```

Environment Setup

GitHub Personal Access Token

Claude Code will need a GitHub token with `repo` scope:

1. Go to: `github.com → Settings → Developer settings → Personal access tokens → Tokens (classic)`
2. Click **Generate new token (classic)**
3. Select scope: ✅ `repo` (full control)
4. Copy the token — you only see it once

Set it in your terminal:

```bash
export GITHUB_TOKEN=ghp_yourTokenHere
```

Or add to your `.env` file:

```
GITHUB_TOKEN=ghp_yourTokenHere
```

> Never commit `.env` or your token to the repo. Add `.env` to `.gitignore`.

Quick Commands for Claude Code

When Claude Code has this file open, say:

You say	Claude does
"Push the site"	Runs `python site_manager.py push`
"Check site health"	Runs `python site_manager.py check`
"Start monitoring"	Runs `python site_manager.py monitor`
"Show monitor log"	Runs `python site_manager.py log`
"Fetch latest from GitHub"	Runs `python site_manager.py fetch`
"Update the hero headline to X, then push"	Edits index.html + pushes
"Add a new pricing tier, push with message 'add tier'"	Edits + `python site_manager.py push "add tier"`

.gitignore

Add this file to keep secrets out of the repo:

```
.env
*.pyc
__pycache__/
.DS_Store
monitor_log.json
```
